

ENSF 380

TOPIC 29: INTRODUCTION TO JAVA GUI PART I

Graphical User Interface

Visualization to facilitate user interaction

AWT – Abstract Window Toolkit

Swing – user interface library built on AWT

AWT and Swing are the Java Foundation Classes (JFC) for creating GUIs

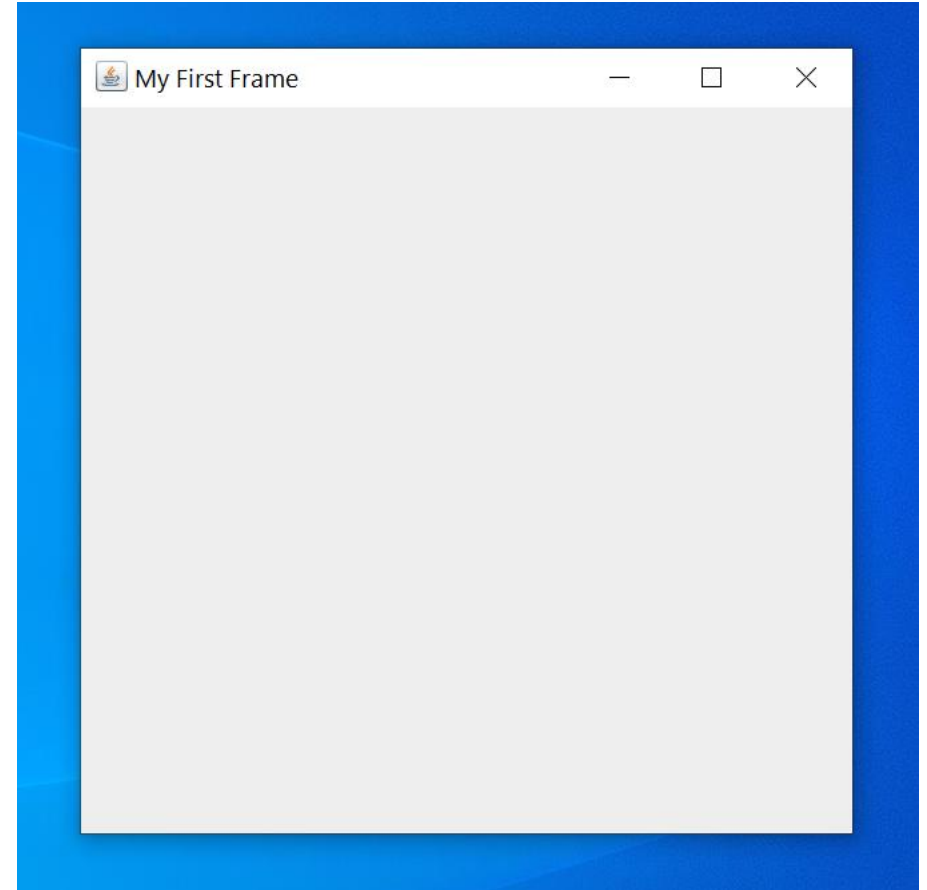
Top-Level Containers

Components are painted on to containers

Top-level containers include JFrame, JDialog, and JApplet

JFrame class creates basic windows

Default appearance provided by system



Creating a Frame

01_MyFirstFrame

```
...
import javax.swing.*;
import java.awt.*;

public class GUIDemo {
    public static void main(String args[]) {

        EventQueue.invokeLater(() -> {
            JFrame frame = new JFrame("My First Frame");
            frame.setSize(400, 400);
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        });
    }
}
```

Tip: javax.swing is included in the standard Java distribution and does not require installing a library.

JFrame Structure

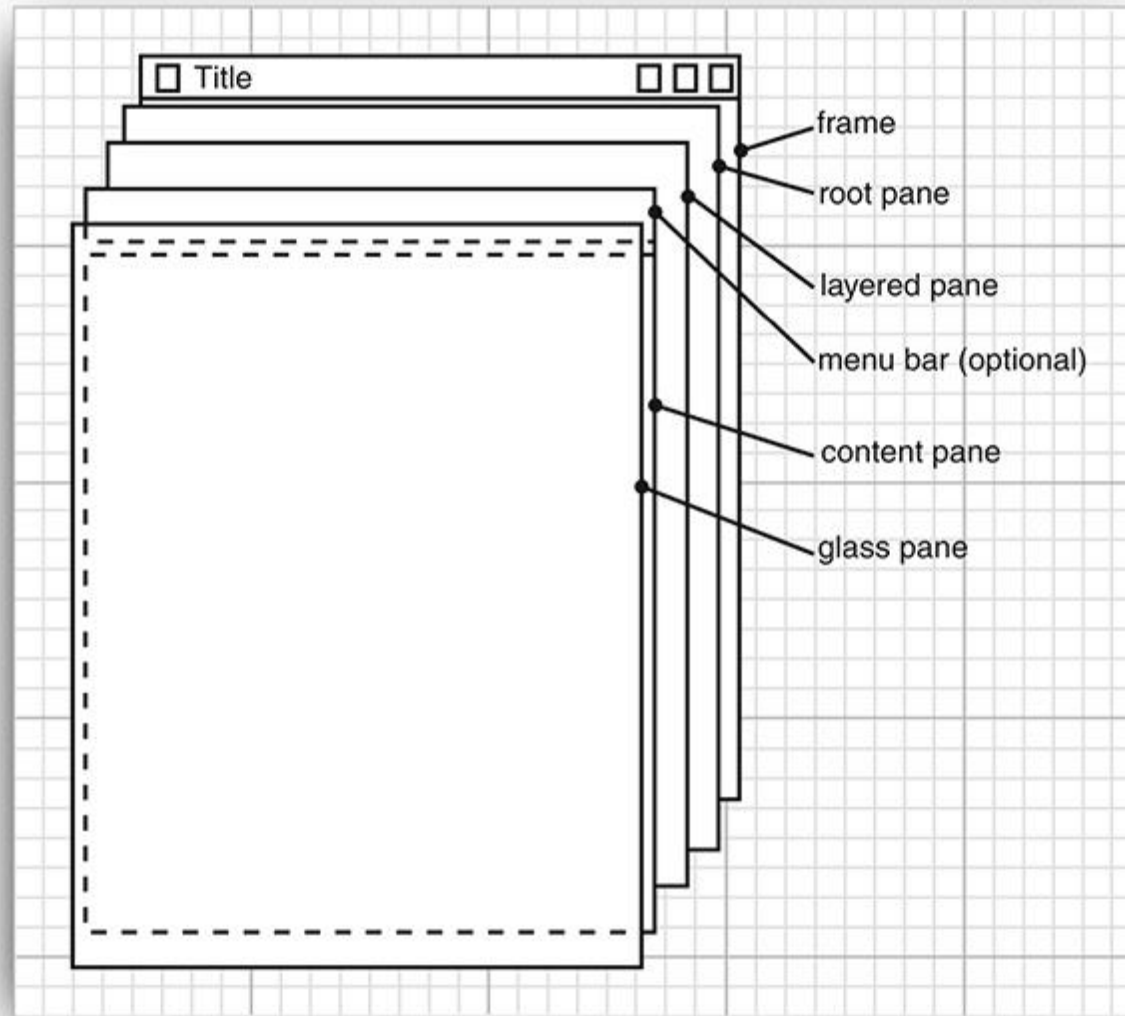


Image from Core Java Vol. 1 - Horstmann

Layout Managers

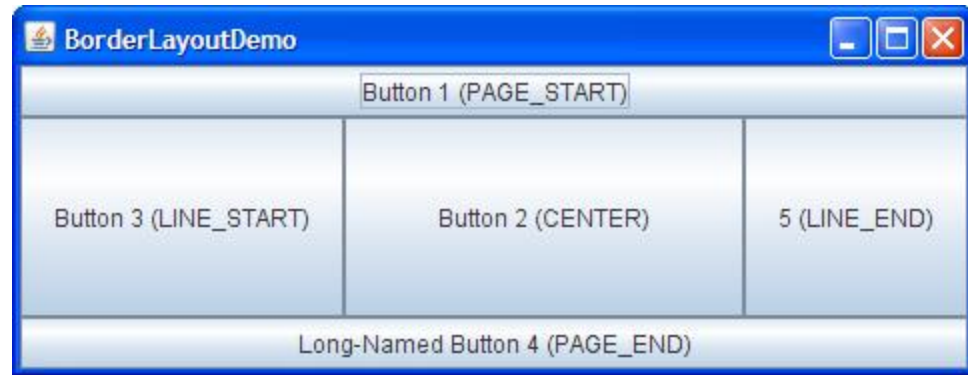
Control positioning and size of components with a container

Use built-in layout managers or create your own

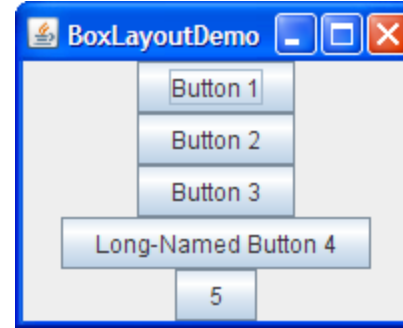
Available layout managers:

- BorderLayout (content pane default)
- BoxLayout
- CardLayout
- FlowLayout
- GridBagLayout
- GridLayout
- GroupLayout
- SpringLayout

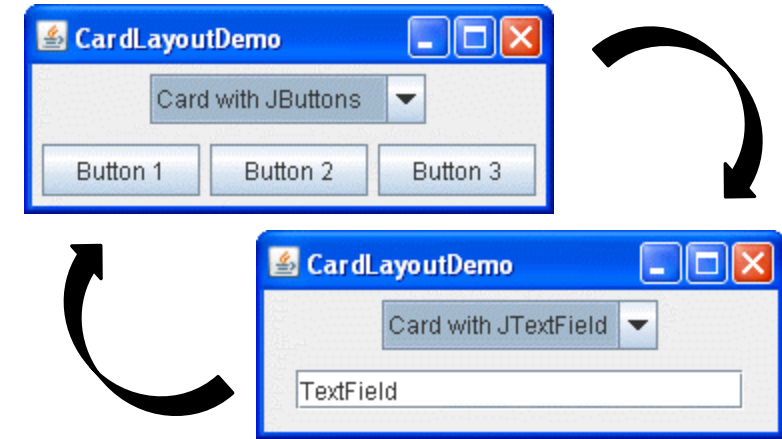
BorderLayout



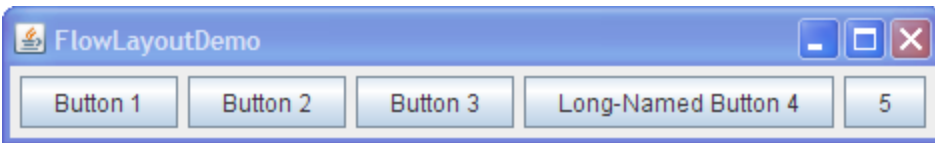
BoxLayout



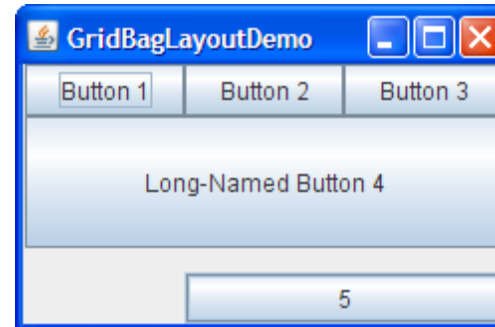
CardLayout



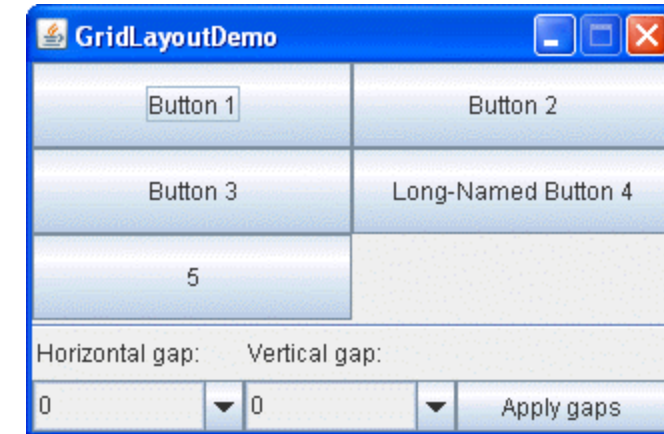
FlowLayout



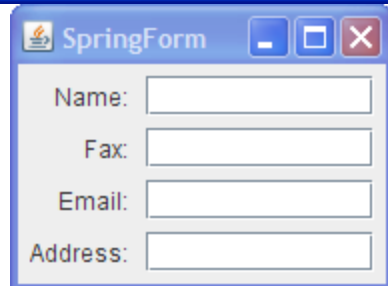
GridBagLayout



GridLayout



SpringLayout



GroupLayout

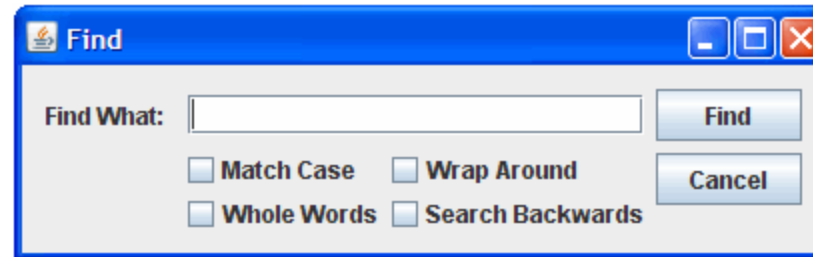


Image from The Java Tutorials - Oracle

Intermediate Containers

Used to simplify positioning of atomic components

Includes JPanel, JSplitPane, JTabbedPane

JPanel class provides space to attach other components

Layout can be defined using a layout manager

Adding Atomic Components

Import the necessary Swing packages

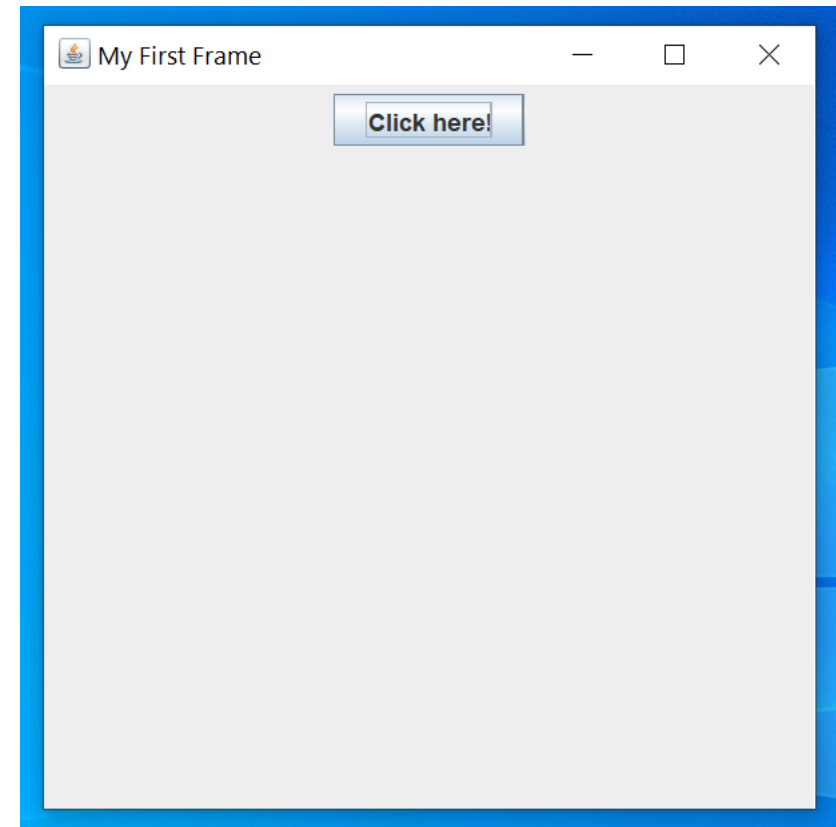
Declare the corresponding panel

Declare the components

Add the components to the panel

Components include:

- JButton
- JCheckBox
- JComboBox
- JLabel
- JList
- JMenu
- JPasswordField
- JSlider
- JTextField
- and more...



Panels and Components

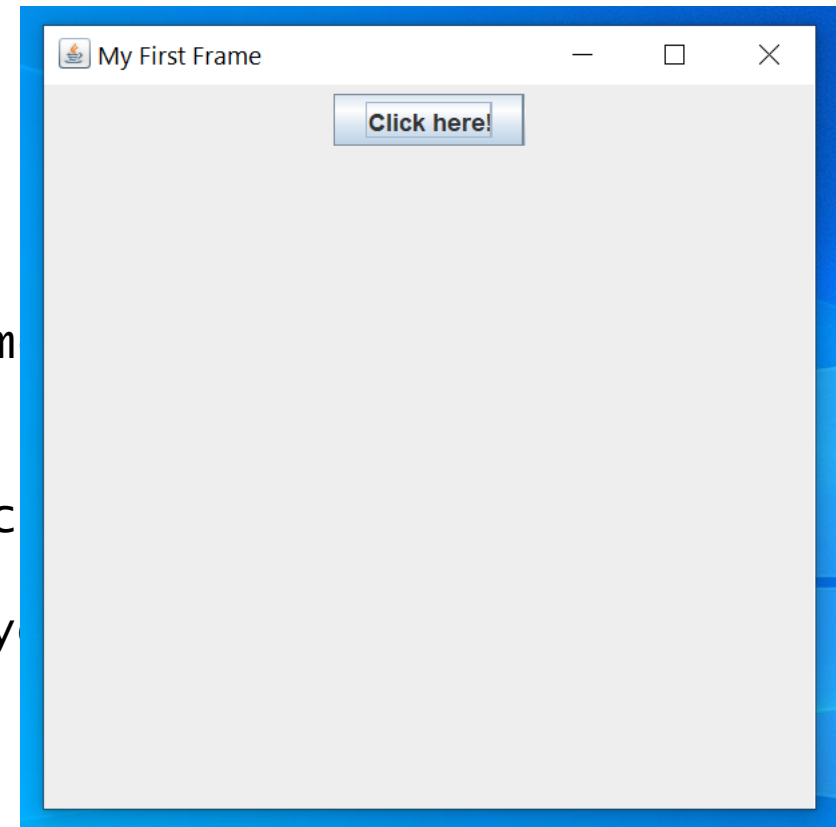
```
...
public class GUIDemo {
    public static void main(String args[]) {

        EventQueue.invokeLater(() -> {
            JFrame frame = new JFrame("My First Frame");
            frame.setSize(400, 400);
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            JPanel buttonsPanel = new JPanel();
            JButton myButton = new JButton("Click here!");
            buttonsPanel.add(myButton);
            frame.getContentPane().add(BorderLayout.NORTH, buttonsPanel);
            frame.setVisible(true);
        });
    }
}
```

Panels and Components

```
public class GUIDemo {  
    public static void main(String args[]) {  
  
        EventQueue.invokeLater(() -> {  
            JFrame frame = new JFrame("My First  
            frame.setSize(400, 400);  
            frame.setDefaultCloseOperation(JFrame.  
  
            JPanel buttonsPanel = new JPanel();  
            JButton myButton = new JButton("Click  
            buttonsPanel.add(myButton);  
            frame.getContentPane().add(BorderLay  
            frame.setVisible(true);  
  
        });  
    }  
}
```



Event Handling

Based on notifications

Objects communicate by “firing” and “handling” event objects

Sent from a single source object to one or more registered listener objects

Different event sources produce different kinds of events

Three involved classes:

- Event source classes (JButton, JTextField, etc.)
- Class event (class that encapsulates the event information)
- Event listeners (receives event object from the source and provides a response)

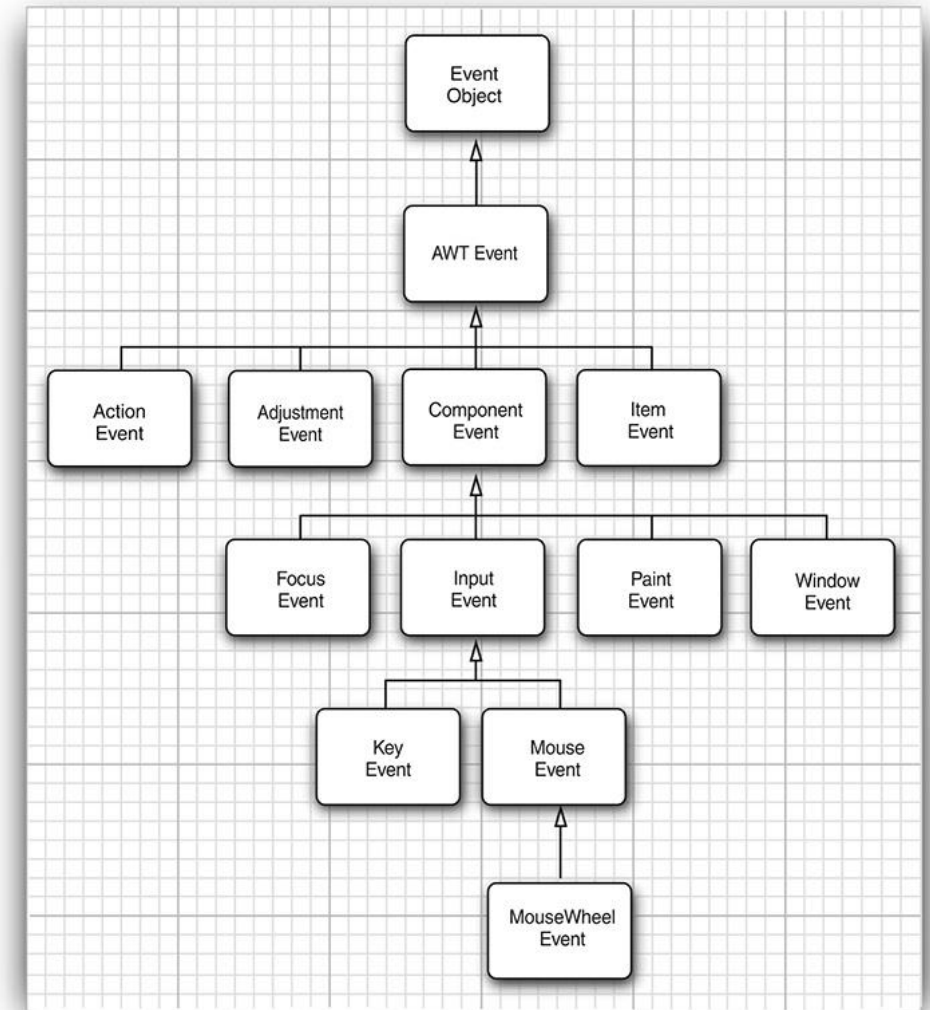


Image from Core Java Vol. 1 - Horstmann

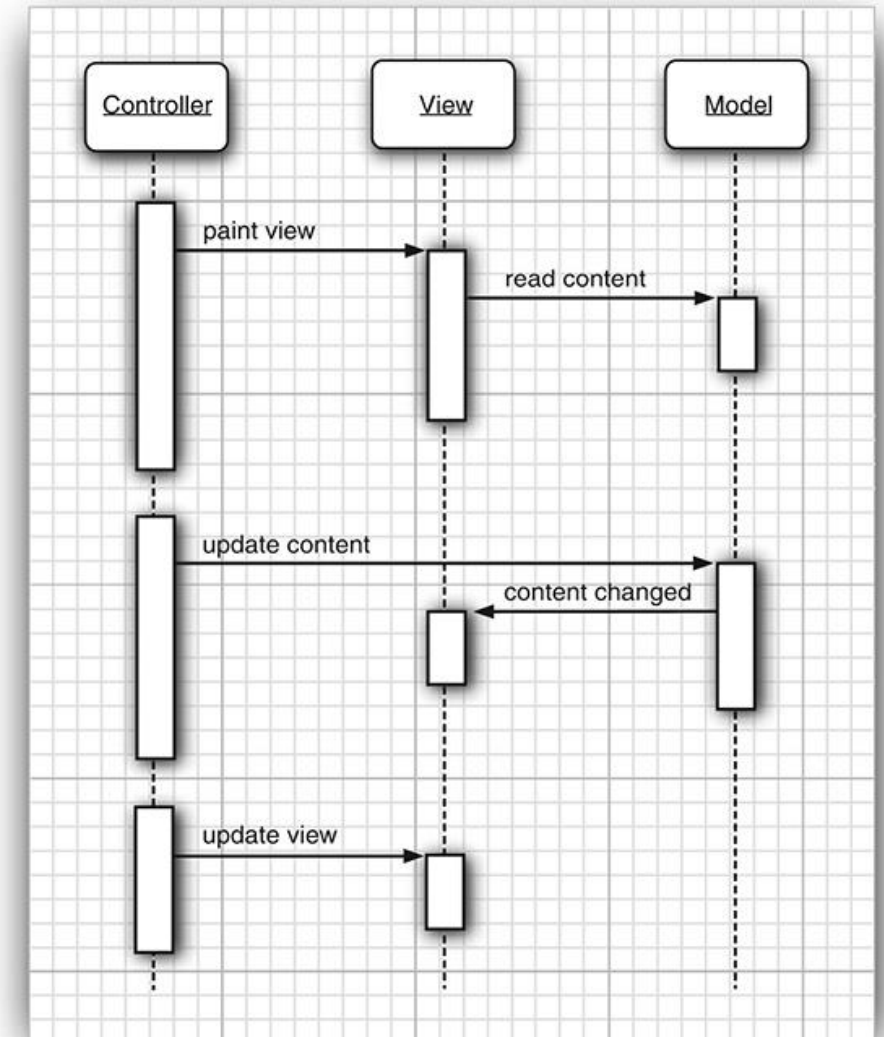
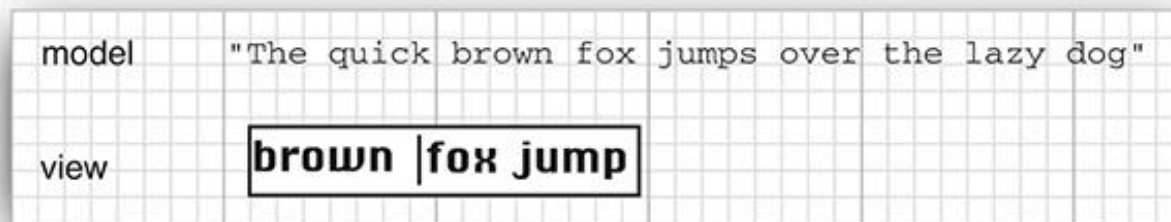
Swing Components and MVC

Component characteristics:

- Content (text, state, etc.)
- Visual appearance (colour, size, etc.)
- Behaviour (reaction to events)

Swing designers use the MVC design pattern

- Model-View-Controller
- Model object stores the content
- View object displays the content
- Controller object handles the user input



Listener Objects

Listener objects wait for events to occur and then respond

Represented by various listener interfaces (MouseListener, ActionListener, etc.)

Event sources are registered to listener objects

Event sources notify all registered listeners when an event occurs

Listener objects use the information encapsulated within the event to determine the response

Implementation:

- Listener class – create your own listener class that implements ActionListener
- Nested inner classes – create/use listener classes within other classes
- Must provide an implementation for interface methods, e.g. `public void actionPerformed(ActionEvent e)`

Handling Events

```
public class GUIDemo implements ActionListener{

    JPanel buttonsPanel = new JPanel();
    JButton myButton = new JButton("Click here!");

    GUIDemo buttonListener = new GUIDemo();
    myButton.addActionListener(buttonListener);
    buttonsPanel.add(myButton);
    frame.getContentPane().add(BorderLayout.NORTH, buttonsPanel);
    frame.setVisible(true);
});

public void actionPerformed(ActionEvent event){
    JOptionPane.showMessageDialog(null, "This button has been pushed");
}
}
```

Handling Events

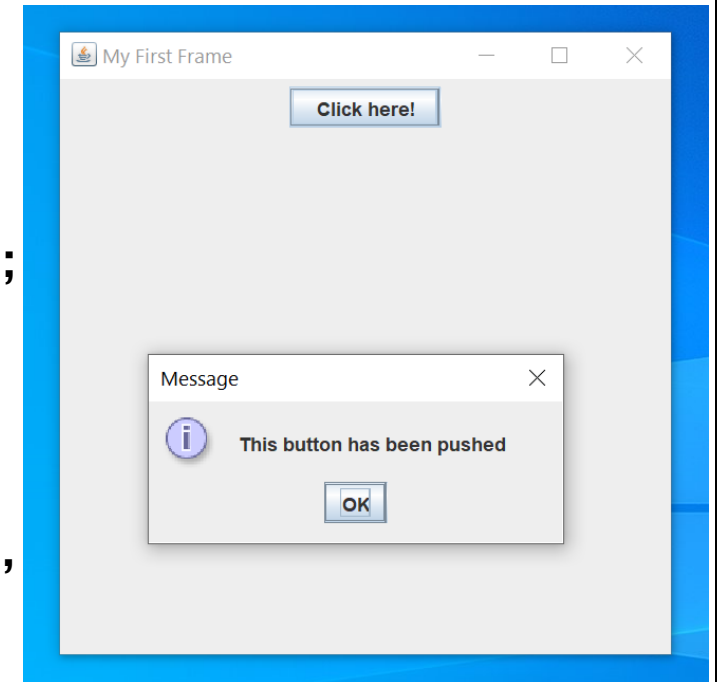
03_EventHandling

```
public class GUIDemo implements ActionListener{

    JPanel buttonsPanel = new JPanel();
    JButton myButton = new JButton("Click here!");

    GUIDemo buttonListener = new GUIDemo();
    myButton.addActionListener(buttonListener);
    buttonsPanel.add(myButton);
    frame.getContentPane().add(BorderLayout.NORTH,
    frame.setVisible(true);
});

public void actionPerformed(ActionEvent event){
    JOptionPane.showMessageDialog(null, "This button has been pushed");
}
}
```



Example: Mouse Events

Notifies when users interact with a component using the mouse

MouseListener interface includes:

- mousePressed
- mouseClicked
- mouseEntered
- mouseExited
- mouseReleased

MouseMotionListener interface includes:

- mouseMoved
- mouseDragged

MouseWheelListener interface includes

- mouseWheelMoved

Summary: Knowledge

Java Foundation Classes (JFC) for GUIs

- AWT and Swing

Containers and components

Layout managers

Event handling

Swing design with MVC

Listener objects